SHERLOCK

Security Review For Vesu

Collaborative Audit Prepared For: **Vesu**
Lead Security Expert(s): **CODESPECT**
Date Audited: **February 6 - February 21, 2026**

Introduction

Vesu is a fully open and permissionless lending protocol built on Starknet. Users can supply crypto assets (earn), borrow crypto assets and build new lending experiences on Vesu without relying on intermediaries. The Vesu lending protocol is not controlled by a governance body and there exists no governance token. Instead, Vesu is built as a public infrastructure giving everyone equal access to all functions and is free for everyone to use.

Scope

Repository: ForgeYields/starknet_vault_kit

Audited Commit: e22d6b0887d01478b983482e235d7843c818f314

Final Commit: babfc20931cb7ba16a86fae0157634026732dcb6

Files:

- packages/vault_allocator/src/adapters/ekubo_adapter/ekubo_adapter.cairo
- packages/vault_allocator/src/adapters/ekubo_adapter/errors.cairo
- packages/vault_allocator/src/adapters/ekubo_adapter/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/avnu_exchange_-decoder_and_sanitizer/avnu_exchange_decoder_and_sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/avnu_exchange_-decoder_and_sanitizer/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/base_decoder_and_-sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/cctp_decoder_and_-sanitizer/cctp_decoder_and_sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/cctp_decoder_and_-sanitizer/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/cctp_middleware_-decoder_and_sanitizer/cctp_middleware_decoder_and_sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/cctp_middleware_-decoder_and_sanitizer/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/decoder_custom_-types.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/defi_spring_decoder_-and_sanitizer/defi_spring_decoder_and_sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/defi_spring_decoder_-and_sanitizer/interface.cairo

- packages/vault_allocator/src/decoders_and_sanitizers/ekubo_adapter_-decoder_and_sanitizer/ekubo_adapter_decoder_and_sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/ekubo_adapter_-decoder_and_sanitizer/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/erc4626_decoder_and_-sanitizer/erc4626_decoder_and_sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/erc4626_decoder_and_-sanitizer/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/extended_decoder_and_-sanitizer/extended_decoder_and_sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/extended_decoder_and_-sanitizer/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/forgeyields_paradex_-decoder_and_sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/fyWBTC_decoder_and_-sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/hyperlane_decoder_and_-sanitizer/hyperlane_decoder_and_sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/hyperlane_decoder_and_-sanitizer/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/hyperlane_middleware_-decoder_and_sanitizer/hyperlane_middleware_decoder_and_sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/hyperlane_middleware_-decoder_and_sanitizer/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/lz_decoder_and_-sanitizer/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/lz_decoder_and_-sanitizer/lz_decoder_and_sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/lz_middleware_decoder_-and_sanitizer/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/lz_middleware_decoder_-and_sanitizer/lz_middleware_decoder_and_sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/multiply_decoder_and_-sanitizer/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/multiply_decoder_and_-sanitizer/multiply_decoder_and_sanitizer.cairo

- packages/vault_allocator/src/decoders_and_sanitizers/paradex_gigavault_-decoder_and_sanitizer/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/paradex_gigavault_-decoder_and_sanitizer/paradex_gigavault_decoder_and_sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/simple_decoder_and_-sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/starkgate_decoder_and_-sanitizer/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/starkgate_decoder_and_-sanitizer/starkgate_decoder_and_sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/starkgate_middleware_-decoder_and_sanitizer/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/starkgate_middleware_-decoder_and_sanitizer/starkgate_middleware_decoder_and_sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/starknet_vault_kit_-decoder_and_sanitizer/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/starknet_vault_kit_-decoder_and_sanitizer/starknet_vault_kit_decoder_and_sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/uncap_decoder_and_-sanitizer/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/uncap_decoder_and_-sanitizer/uncap_decoder_and_sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/vesu_decoder_and_-sanitizer/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/vesu_decoder_and_-sanitizer/vesu_decoder_and_sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/vesu_v2_decoder_and_-sanitizer/interface.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/vesu_v2_decoder_and_-sanitizer/vesu_v2_decoder_and_sanitizer.cairo
- packages/vault_allocator/src/decoders_and_sanitizers/vesu_v2_specific_-decoder_and_sanitizer.cairo
- packages/vault_allocator/src/integration_interfaces/avnu.cairo
- packages/vault_allocator/src/integration_interfaces/cctp.cairo
- packages/vault_allocator/src/integration_interfaces/ekubo.cairo
- packages/vault_allocator/src/integration_interfaces/hyperlane.cairo
- packages/vault_allocator/src/integration_interfaces/lz.cairo

- packages/vault_allocator/src/integration_interfaces/paradex_gigavault.cairo
- packages/vault_allocator/src/integration_interfaces/pragma.cairo
- packages/vault_allocator/src/integration_interfaces/starkgate.cairo
- packages/vault_allocator/src/integration_interfaces/vesu_v1.cairo
- packages/vault_allocator/src/integration_interfaces/vesu_v2.cairo
- packages/vault_allocator/src/lib.cairo
- packages/vault_allocator/src/manager/errors.cairo
- packages/vault_allocator/src/manager/interface.cairo
- packages/vault_allocator/src/manager/manager.cairo
- packages/vault_allocator/src/merkle_tree/base.cairo
- packages/vault_allocator/src/merkle_tree/integrations/avnu.cairo
- packages/vault_allocator/src/merkle_tree/integrations/cctp.cairo
- packages/vault_allocator/src/merkle_tree/integrations/defi_spring.cairo
- packages/vault_allocator/src/merkle_tree/integrations/ekubo_adapter.cairo
- packages/vault_allocator/src/merkle_tree/integrations/erc4626.cairo
- packages/vault_allocator/src/merkle_tree/integrations/extended.cairo
- packages/vault_allocator/src/merkle_tree/integrations/hyperlane.cairo
- packages/vault_allocator/src/merkle_tree/integrations/lz.cairo
- packages/vault_allocator/src/merkle_tree/integrations/starkgate.cairo
- packages/vault_allocator/src/merkle_tree/integrations/starknet_vault_kit_-strategies.cairo
- packages/vault_allocator/src/merkle_tree/integrations/vesu_v1.cairo
- packages/vault_allocator/src/merkle_tree/integrations/vesu_v2.cairo
- packages/vault_allocator/src/merkle_tree/registery.cairo
- packages/vault_allocator/src/middlewares/avnu_middleware/avnu_-middleware.cairo
- packages/vault_allocator/src/middlewares/avnu_middleware/errors.cairo
- packages/vault_allocator/src/middlewares/avnu_middleware/interface.cairo
- packages/vault_allocator/src/middlewares/base_middleware/base_-middleware.cairo
- packages/vault_allocator/src/middlewares/base_middleware/errors.cairo
- packages/vault_allocator/src/middlewares/base_middleware/interface.cairo

- packages/vault_allocator/src/middlewares/cctp_middleware/cctp_-middleware.cairo
- packages/vault_allocator/src/middlewares/cctp_middleware/errors.cairo
- packages/vault_allocator/src/middlewares/cctp_middleware/interface.cairo
- packages/vault_allocator/src/middlewares/hyperlane_middleware/errors.cairo
- packages/vault_allocator/src/middlewares/hyperlane_middleware/hyperlane_-middleware.cairo
- packages/vault_allocator/src/middlewares/hyperlane_-middleware/interface.cairo
- packages/vault_allocator/src/middlewares/lz_middleware/errors.cairo
- packages/vault_allocator/src/middlewares/lz_middleware/interface.cairo
- packages/vault_allocator/src/middlewares/lz_middleware/lz_middleware.cairo
- packages/vault_allocator/src/middlewares/paradex_gigavault_-middleware/interface.cairo
- packages/vault_allocator/src/middlewares/paradex_gigavault_-middleware/paradex_gigavault_middleware.cairo
- packages/vault_allocator/src/middlewares/starkgate_middleware/errors.cairo
- packages/vault_allocator/src/middlewares/starkgate_middleware/interface.cairo
- packages/vault_allocator/src/middlewares/starkgate_middleware/starkgate_-middleware.cairo
- packages/vault_allocator/src/periphery/price_router/errors.cairo
- packages/vault_allocator/src/periphery/price_router/interface.cairo
- packages/vault_allocator/src/periphery/price_router/price_router.cairo
- packages/vault_allocator/src/periphery/price_router_vesu/errors.cairo
- packages/vault_allocator/src/periphery/price_router_vesu/interface.cairo
- packages/vault_allocator/src/periphery/price_router_vesu/price_router_-vesu.cairo
- packages/vault_allocator/src/pods/base_pod.cairo
- packages/vault_allocator/src/pods/components/asset_transfer_pod.cairo
- packages/vault_allocator/src/pods/components/errors.cairo
- packages/vault_allocator/src/pods/components/interface.cairo
- packages/vault_allocator/src/vault_allocator/errors.cairo
- packages/vault_allocator/src/vault_allocator/interface.cairo
- packages/vault_allocator/src/vault_allocator/vault_allocator.cairo

- packages/vault/src/lib.cairo
- packages/vault/src/redeem_request/errors.cairo
- packages/vault/src/redeem_request/redeem_request.cairo
- packages/vault/src/vault/vault.cairo

Repository: vesuxyz/vesu-vaults

Audited Commit: d252d385cd0efefce3401ce90e15a725d4efba2e

Final Commit: 879dcb283be7829da0b1f840f92ad1c4b8898118

Files:

- src/aum_provider/aum_provider.cairo
- src/aum_provider/errors.cairo
- src/aum_provider/interface.cairo
- src/lib.cairo
- src/merkle_tree/base.cairo
- src/merkle_tree_creator.cairo
- src/merkle_tree/integrations/avnu.cairo
- src/merkle_tree/integrations/cctp.cairo
- src/merkle_tree/integrations/erc4626.cairo
- src/merkle_tree/integrations/hyperlane.cairo
- src/merkle_tree/integrations/lz.cairo
- src/merkle_tree/integrations/starkgate.cairo
- src/merkle_tree/integrations/starknet_vault_kit_strategies.cairo
- src/merkle_tree/integrations/vesu_v2.cairo
- src/merkle_tree/registry.cairo
- src/periphery_interfaces/avnu_router_middleware.cairo
- src/periphery_interfaces/base_middleware.cairo
- src/periphery_interfaces/cctp_middleware.cairo
- src/periphery_interfaces/hyperlane_middleware.cairo
- src/periphery_interfaces/lz.cairo
- src/periphery_interfaces/lz_middleware.cairo
- src/periphery_interfaces/manager.cairo

- src/periphery_interfaces/pragma.cairo
- src/periphery_interfaces/price_router.cairo
- src/periphery_interfaces/starkgate.cairo
- src/periphery_interfaces/starkgate_middleware.cairo
- src/periphery_interfaces/starknet_vault_kit.cairo
- src/periphery_interfaces/vault_allocator.cairo
- src/periphery_interfaces/vesu_v2.cairo
- src/price_router/errors.cairo
- src/price_router/interface.cairo
- src/price_router/price_router.cairo
- src/rewards_handler/errors.cairo
- src/rewards_handler/interface.cairo
- src/rewards_handler/rewards_handler.cairo
- src/vault_factory/interface.cairo
- src/vault_factory/vault_factory.cairo
- src/vault_governor/errors.cairo
- src/vault_governor/interface.cairo
- src/vault_governor/vault_governor.cairo

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	3	8

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue M-1: Ekubo adapter fee collection flag prevents collection of pending fees on full withdrawal [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-vesu-feb-4th/issues/134

Summary

The `withdraw_liquidity(...)` function in the Ekubo adapter does not collect pending fees when withdrawing liquidity. When the withdrawal amount equals total liquidity, the adapter clears the NFT identifier, making unclaimed fees irrecoverable.

Vulnerability Detail

The `withdraw_liquidity(...)` function calls the Ekubo contract's `withdraw(...)` function with the fee collection flag hardcoded to `false`. This means any pending fees accumulated in the position are not collected during withdrawal. If the withdrawal amount equals total liquidity, the adapter clears the `contract_nft_id` storage variable. Since unclaimed fees remain tied to the NFT and the `contract_nft_id` is discarded, these fees become permanently unrecoverable.

Impact

Pending fees that have accumulated in the Ekubo position are permanently lost when withdrawing the full liquidity amount. This results in direct loss of yield for the vault.

Code Snippet

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/starknet_vault_kit/packages/vault_allocator/src/adapters/ekubo_adapter/ekubo_adapter.cairo#L150

Tool Used

Manual Review

Recommendation

If the withdrawal amount equals the total liquidity, the fee collection flag should be set to true to collect pending fees.

Discussion

0xSacha

https://github.com/ForgeYields/starknet_vault_kit/commit/ade42e64eb5ea2004499fafa1479a09bc25c809c

Kalogerone

Fixed

Issue M-2: Bridge middlewares are prone to DOS from small donations [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-vesu-feb-4th/issues/137

Summary

The bridge middleware contracts implement a function that initiates the bridging process. This function has a check that the contract's `token_to_claim` balance must be 0. This check is intended to prevent multiple simultaneous cross-chain swaps of `token_to_claim` from interfering with each other. However, anyone can send 1 wei of the token and completely DOS the contract.

Vulnerability Detail

This check is enforced in all of the bridge middlewares:

With just 1 wei the contract gets compeleted DOS'd. The function that initiates the bridging process will always revert and there is no token rescuing mechanism to remove this 1 wei from the contract.

Impact

The bridge middleware contracts can get DOS'd by a small donation.

Code Snippet

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/starknet_vault_kit/packages/vault_allocator/src/middlewares/lz_middleware/lz_middleware.cairo#L101

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/starknet_vault_kit/packages/vault_allocator/src/middlewares/starkgate_middleware/starkgate_middleware.cairo#L98

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/starknet_vault_kit/packages/vault_allocator/src/middlewares/hyperlane_middleware/hyperlane_middleware.cairo#L101

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/starknet_vault_kit/packages/vault_allocator/src/middlewares/cctp_middleware/cctp_middleware.cairo#L105

Tool Used

Manual Review

Recommendation

Consider implementing a `pendingToken(address => bool)` mapping instead of checking balances, allowing only one request per `token_to_claim` to exist at the same time.

Discussion

0xSacha

https://github.com/ForgeYields/starknet_vault_kit/commit/404e08dc9db88cd95bad47486df06f48cea6b8de

Kalogerone

Fixed

Issue M-3: Bridge middlewares claiming can be griefed [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-vesu-feb-4th/issues/138

Summary

The bridges' middleware `claim_token(...)` function allows anyone to call it and clear a request's `pending_balance`. This can be exploited by users when there are multiple requests for a `token_to_claim`.

Vulnerability Detail

When there are multiple requests for a `token_to_claim` anyone can call the `claim_token(...)` function and use the current `token_to_claim` balance for any request he desires. For example:

There are 2 bridging requests:

1. 1000 `USDT` for ETH
2. 100 `USDC` for ETH

The 1st request gets completed. Users can now call the `claim_token(...)` function for the 2nd request, which resets the mapping. When the 2nd request gets completed, the amount of ETH will be too low to call `claim_token(...)` on the 1st request.

Additionally, if both requests get completed at the same time, it's only possible to call `claim_token(...)` on only one of them.

Impact

Users can DOS certain requests and some requests can't be cleared, blocking the creation of similar requests.

Code Snippet

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/starknet_vault_kit/packages/vault_allocator/src/middlewares/starkgate_middleware/starkgate_middleware.cairo#L143

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/starknet_vault_kit/packages/vault_allocator/src/middlewares/lz_middleware/lz_middleware.cairo#L166

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/starknet_vault_kit/packages/vault_allocator/src/middlewares/hyperlane_middleware/hyperlane_middleware.cairo#L169

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/starknet_vault_kit/packages/vault_allocator/src/middlewares/cctp_middleware/cctp_middleware.cairo#L167

Tool Used

Manual Review

Recommendation

Consider implementing a `pendingToken(address => bool)` mapping, allowing only one request per `token_to_claim` to exist at the same time.

Discussion

0xSacha

https://github.com/ForgeYields/starknet_vault_kit/commit/404e08dc9db88cd95bad47486df06f48cea6b8de

Kalogerone

Fixed

Issue L-1: Bridge middlewares calculate the slippage at the time of claiming, not at the time of the swap [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-02-vesu-feb-4th/issues/136

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The bridge middleware contracts implement two functions, a function that initiates the bridging process and a `claim_token(....)` function. The `claim_token(...)` function uses the current prices of the tokens to calculate the slippage and the minimum output amount. If the contract's token balance is not greater than this amount, the transaction reverts. However, the current prices of the tokens don't reflect the prices at the time of the swap.

Vulnerability Detail

Price fluctuations between the swap and the completion of the bridging process can make the `claim_token(....)` function revert. At these cases funds are temporarily stuck. Admins must manually transfer tokens to the middleware contract to increase its token balance, so the `claim_token(....)` function will not revert.

Impact

The `claim_token(....)` function can unnecessarily revert and admins must manually transfer funds to rescue the locked tokens.

Code Snippet

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/starknet_vault_kit/packages/vault_allocator/src/middlewares/starkgate_middleware/starkgate_middleware.cairo#L143

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/starknet_vault_kit/packages/vault_allocator/src/middlewares/lz_middleware/lz_middleware.cairo#L166

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/starknet_vault_kit/packages/vault_allocator/src/middlewares/hyperlane_middleware/hyperlane_middleware.cairo#L169

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/starknet_vault_kit/packages/vault_allocator/src/middlewares/cctp_middleware/cctp_middleware.cairo#L167

Tool Used

Manual Review

Recommendation

Consider one of the following solutions:

1. Sending a cross-chain message and accurately set the expected tokens to claim.
2. Setting a less strict slippage to allow for high price fluctuations (currently set at 1% in tests).

Discussion

0xSacha

Tokens are used to be correlated. Acknowledged

Kalogerone

Acknowledged

Issue L-2: Strategy permission control lacks flexibility [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-02-vesu-feb-4th/issues/139

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The `set_vault_configs(...)` function in `vault_governor.cairo` overwrites the entire strategy configuration on each call, preventing independent permission assignments to different strategies.

Vulnerability Detail

The `set_vault_configs(...)` function allows the governor to configure various strategies including `vesu_v2_configs`, `starknet_vault_kit_strategies`, and others. However, the permission control mechanism lacks flexibility because each call invokes `vault_aum_provider.set_vault_config(...)`, which overwrites the previous strategy configuration entirely.

This design makes it impossible to assign different permission sets to different strategies. For example, it's not possible to assign `vesuv2` permissions to one strategy and `starknet_vault_kit_strategies` permissions to another. All permissions must follow a subset relationship where one strategy can have `vesuv2`, and another strategy can have `vesuv2 + starknet_vault_kit_strategies`, but they cannot have completely independent permission sets.

Impact

This limitation reduces the flexibility of strategy management. In scenarios where different strategies require independent permission sets that don't follow a subset relationship, the protocol cannot accommodate such configurations.

Code Snippet

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/vesu-vaults/src/vault_governor/vault_governor.cairo#L1094

Tool Used

Manual Review

Recommendation

Consider redesigning the permission system to allow independent strategy configurations.

Discussion

nbundi

Acknowledged.

Kalogerone

Acknowledged

Issue L-3: Bridging middleware cannot be reused across configurations [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-vesu-feb-4th/issues/140

Summary

During vault creation, if two configurations share the same middleware contract address, the second ownership transfer will fail because the owner has already been transferred.

Vulnerability Detail

During vault creation, the `create_vault(...)` function initializes various middleware configurations and transfers ownership to the `vault_governor`. The initialization loops through `starkgate_configs`, `hyperlane_configs`, `lz_configs`, and `cctp_configs`, transferring ownership of each middleware to the vault governor.

However, if two configurations share the same middleware contract address, the second ownership transfer will fail because the owner has already been transferred to the `vault_governor` during the first iteration. This prevents middleware reuse across different configurations.

Impact

This design prevents efficient reuse of middleware contracts across different bridge tokens, potentially leading to unnecessary contract deployments and increased gas costs.

Code Snippet

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/vesu-vaults/src/vault_factory/vault_factory.cairo#L367
https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/vesu-vaults/src/vault_factory/vault_factory.cairo#L377
https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/vesu-vaults/src/vault_factory/vault_factory.cairo#L387
https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/vesu-vaults/src/vault_factory/vault_factory.cairo#L397

Tool Used

Manual Review

Recommendation

Consider checking if ownership has already been transferred before attempting another transfer, or document this limitation clearly so integrators know each configuration requires a unique middleware instance.

Discussion

nbundi

Fixed https://github.com/vesuxyz/vesu-vaults/commit/657383e3cb798207f93fc5a097a422de56db790d

Kalogerone

Fixed

Issue L-4: Redundant approval operation in HyperlaneMiddleware [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-vesu-feb-4th/issues/142

Summary

The `bridge(...)` function performs an unnecessary approval of `token_to_bridge` to itself before calling `transfer_remote`.

Vulnerability Detail

The `bridge(...)` function in `HyperlaneMiddleware` performs an approval of `token_to_bridge` to itself before calling `transfer_remote`. This approval step appears to be redundant since calling `transfer_remote` on the token contract does not require the caller to approve itself. The tests still pass even if this approval line is removed, similar to how OFT operations work.

Impact

This redundancy increases gas costs unnecessarily and adds complexity to the code without providing any benefit.

Code Snippet

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/starknet_vault_kit/packages/vault_allocator/src/middlewares/hyperlane_middleware/hyperlane_middleware.cairo#L146

Tool Used

Manual Review

Recommendation

Remove the redundant approval operation to save gas and simplify the code.

Discussion

0xSacha

https://github.com/ForgeYields/starknet_vault_kit/commit/182f4439ae9efcacafa71b38e3d525eec216ba63

Kalogerone

Fixed

Issue L-5: Missing `claim_all_coll_gains` operation in UncapDecoderAndSanitizer [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-vesu-feb-4th/issues/143

Summary

The UncapDecoderAndSanitizer lacks a `claim_all_coll_gains` operation to claim rewards stored in `stashed_coll`.

Vulnerability Detail

In `UncapDecoderAndSanitizer`, only the `provide_to_sp` and `withdraw_from_sp` functions exist. In the Uncap StabilityPool implementation, if `do_claim` is false during withdrawal, collateral rewards are stored in `stashed_coll`. However, `UncapDecoderAndSanitizer` lacks a function to claim the rewards in `stashed_coll` separately.

Impact

If a full withdrawal is made with `do_claim` set to false, the rewards in `stashed_coll` cannot be claimed separately and require another call to `provide_to_sp` on Uncap to retrieve.

Code Snippet

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/starknet_vault_kit/packages/vault_allocator/src/decoders_and_sanitizers/uncap_decoder_and_sanitizer/uncap_decoder_and_sanitizer.cairo#L20

Tool Used

Manual Review

Recommendation

Add a claim_all_coll_gains operation to the `UncapDecoderAndSanitizer` to enable claiming of stashed collateral rewards.

Discussion

0xSacha

https://github.com/ForgeYields/starknet_vault_kit/commit/924f62d733c32d77f5a89ea46e21c6da092026f3

Kalogerone

Fixed

Issue L-6: Setting a manager in the Vault Allocator doesn't emit an event [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-02-vesu-feb-4th/issues/144

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The `set_manager(...)` function in the `vault_allocator` contract doesn't emit an event.

Vulnerability Detail

Impact

No event emission when setting a manager.

Code Snippet

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/starknet_vault_kit/packages/vault_allocator/src/vault_allocator/vault_allocator.cairo#L85

Tool Used

Manual Review

Recommendation

Add an event emission at the end of the function.

Discussion

0xSacha

Acknowledged. The set_manager function does not emit an event. This is a known limitation and does not impact the protocol's security. An event emission may be added in a future update for improved off-chain tracking, but the curator is expected to monitor manager changes through other means.

Kalogerone

Acknowledged

Issue L-7: The Avnu Middleware initializes the owner twice [RESOLVED]

Source: https://github.com/sherlock-audit/2026-02-vesu-feb-4th/issues/145

Summary

The `avnu_middleware` contract initializes the `ownable` component twice.

Vulnerability Detail

The `constructor(...)` of the contract initializes the owner. However, the `base_middleware` also does this when the `initialize_base_middleware(...)` gets called:

Impact

The `ownable` component gets initialized twice.

Code Snippet

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/starknet_vault_kit/packages/vault_allocator/src/middlewares/avnu_middleware/avnu_middleware.cairo#L76

Tool Used

Manual Review

Recommendation

Remove the initialization at the `constructor(...)`.

Discussion

0xSacha

https://github.com/ForgeYields/starknet_vault_kit/commit/8cb863a7aa46c703a538b2e811bbf41ead2be304

Kalogerone

Fixed

Issue L-8: The LZ Middleware doesn't verify some amounts directly with the OFT contract [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-02-vesu-feb-4th/issues/146

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The `lz_middleware` contract implements a `send(...)` function which initiates the bridging process. The LayerZero team recommends to implement additional functionality for smoother interactions.

Vulnerability Detail

It is possible that the bridged token has different decimals between chains. It is recommended to call quote_oft(...) to verify that small amounts don't get lost due to this.

Additionally, the quote_send(...) function can be called to retrieve the exact fee needed for the bridging. Currently, the bridging fee is a user input and is prone to mistakes without it being verified.

Impact

It's possible to lose dust amounts due to decimal differences or wrong fee amounts to be used.

Code Snippet

https://github.com/sherlock-audit/2026-02-vesu-feb-4th/blob/38ca4effc2a0bd6f23cf712efed532d13388d871/starknet_vault_kit/packages/vault_allocator/src/middlewares/lz_middleware/lz_middleware.cairo#L101

Tool Used

Manual Review

Recommendation

Consider calling both of the OFT functions mentioned.

Discussion

0xSacha

Acknowledged. Assets currently bridged via LayerZero on starknet (WBTC) share the same decimals across chains, so no dust loss occurs in practice. If a new asset with differing decimals is introduced, the curator is expected to account for this and avoid amounts that could result in dust loss. Regarding the messaging fee, it is provided as an argument by the curator, who is expected to fetch the correct value off-chain before submitting the transaction.

Kalogerone

Acknowledged

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.